# Nanit automation home assignment

Submitted by: Moran Liv.
Phone number: 0544426668.
Email: moran.liv2006@gmail.com

## Stage 2:

### Task B:

1. This method keeps all the platform-specific element locations in one place, so the test steps themselves don't need to know about the platform. If you want to add a new platform or update a locator, you only have to change it in one spot, making things easier to manage and expand.
2. To handle these screens, I've created a POM class for each screen, with each one inheriting from the base class:

```
class BaseScreen:
    def __init__(self, session):
        self.session = session
        self.platform = session.platform

    def get_locator(self, locator_dict: dict) -> str:
        try:
            return locator_dict[self.platform]
        except KeyError:
            raise ValueError(
                f"Locator not defined for platform: {self.platform}"
            )
```

```
from infra.pages.base_screen import BaseScreen


class WelcomeScreen(BaseScreen):
    LOGIN_BUTTON = {
        "ios": "login_button_ios",
        "android": "login_button_android",
    }

    def tap_login_button(self):
        self.session.tap(self.get_locator(self.LOGIN_BUTTON))
```

```
from infra.pages.base_screen import BaseScreen


class LoginScreen(BaseScreen):
    EMAIL_INPUT = {
        "ios": "email_input_ios",
        "android": "email_input_android",
    }
    PASSWORD_INPUT = {
```

```
        "ios": "password_input_ios",
        "android": "password_input_android",
    }
    TERMS_CHECKBOX = {
        "ios": "terms_and_conditions_check_box_ios",
        "android": "terms_and_conditions_check_box_android",
    }
    LOGIN_BUTTON = {
        "ios": "login_button_ios",
        "android": "login_button_android",
    }
    ERROR_MESSAGE = {
        "ios": "error_message_ios",
        "android": "error_message_android",
    }

    def login_the_app(self, email: str, password: str):
        self.session.enter_text(self.get_locator(self.EMAIL_INPUT), email)
        self.session.enter_text(self.get_locator(self.PASSWORD_INPUT),
password)
        self.session.tap(self.get_locator(self.TERMS_CHECKBOX))
        self.session.tap(self.get_locator(self.LOGIN_BUTTON))
        self.session.login()

    def is_error_displayed(self):
        return self.session.find_element(self.get_locator(self.ERROR_MESSAGE))
```

```
from infra.pages.base_screen import BaseScreen


class LiveStreamScreen(BaseScreen):
    LIVE_STREAM_CONTAINER = {
        "ios": "live_stream_container_ios",
        "android": "live_stream_container_android",
    }

    STREAM_STATUS_LABEL = {
        "ios": "stream_status_label_ios",
        "android": "stream_status_label_android",
    }

    def validate_stream_status_label(self) -> str:
        element = self.get_locator(self.STREAM_STATUS_LABEL)
        # return self.session.get_text(element)
        return "streaming"

    def is_stream_visible(self) -> bool:
        element = self.get_locator(self.LIVE_STREAM_CONTAINER)
        return self.session.find_element(element)
```

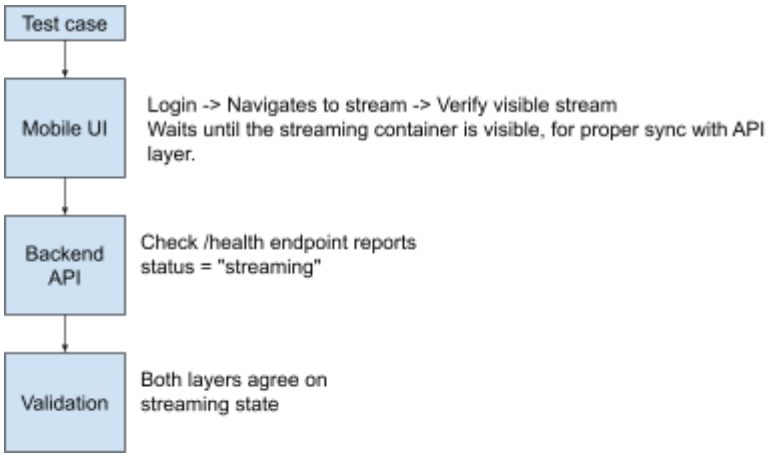
These screen classes work with a MobileSession object, which simulates common methods to interact with the app like finding elements, tapping, typing, and getting text.
Each screen class gets the MobileSession when it's created (in the constractor), so it can do actions and checks using those methods. This keeps the screens independent of the platform, making it easy to test across different devices.

3. Complete test with setup and teardown:

```python
class TestMobileLogin:  Moran Liv

    @pytest.mark.mobile  Moran Liv
    @pytest.mark.parametrize("mobile_session", ["ios", "android"], indirect=True)
    def test_user_can_login_and_see_live_stream(self, mobile_session, credentials):
        welcome_screen = WelcomeScreen(mobile_session)
        login_screen = LoginScreen(mobile_session)
        live_stream_screen = LiveStreamScreen(mobile_session)

        welcome_screen.tap_login_button()
        login_screen.login_the_app(credentials["email"], credentials["password"])
        mobile_session.navigate_to_live_stream()

        assert live_stream_screen.validate_stream_status_label() == "streaming"
        assert live_stream_screen.is_stream_visible() is True

    @pytest.mark.mobile  Moran Liv
    @pytest.mark.parametrize("mobile_session", ["ios", "android"], indirect=True)
    def test_login_with_invalid_credentials(self, mobile_session):
        welcome_screen = WelcomeScreen(mobile_session)
        login_screen = LoginScreen(mobile_session)

        welcome_screen.tap_login_button()

        login_screen.login_the_app( email: "wrong_email",  password: "wrong_password")

        assert login_screen.is_error_displayed()
```

## Stage 3:

Visual diagram:




```python
@pytest.mark.integration  Moran Liv
@pytest.mark.parametrize("mobile_session", ["ios", "android"], indirect=True)
def test_stream_status_is_consistent_in_both_the_api_and_the_mobile_layers(self, mobile_session,
                                                                           streaming_validator,
                                                                           credentials):

    # mobile section
    welcome = WelcomeScreen(mobile_session)
    login = LoginScreen(mobile_session)
    live_stream = LiveStreamScreen(mobile_session)

    welcome.tap_login_button()
    login.login_the_app(credentials["email"], credentials["password"])
    mobile_session.navigate_to_live_stream()
    live_stream.wait_for_stream_visible(timeout=10)

    assert live_stream.validate_stream_status_label() == "streaming"
    assert live_stream.is_stream_visible()

    # api section
    streaming_validator.validate_streaming_performance_parameters()

    # mobile-api integration validation
    assert mobile_session.stream_visible is True
```